



**SCHOOL OF
COMPUTING**

DESIGN AND ANALYSIS OF ALGORITHMS

LAB WORKBOOK

WEEK - 6

NAME : SANTHOSH A

ROLL NUMBER : CH.SC.U4CSE24142

CLASS : CSE-B

Question 1: Using the Quick Select technique, find the k-th smallest element in the following list. Assume the last element as the pivot in each partition.

157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117

CODE:

```
//CH.SC.U4CSE24142 - SANTHOSH A
#include <stdio.h>
int partition(int arr[], int low, int high) {
    int pivot=arr[high];
    int i=low;
    for(int j=low;j<high;j++){
        if(arr[j]<pivot){
            int temp=arr[i];
            arr[i]=arr[j];
            arr[j]=temp;
            i++;
        }
    }
    int temp=arr[i];
    arr[i]=arr[high];
    arr[high]=temp;
    return i;
}
int quickSelect(int arr[], int low, int high, int k) {
    if(low<=high){
        int pivotIndex=partition(arr, low, high);
        if(pivotIndex==k)
            return arr[pivotIndex];
        else if(pivotIndex>k)
            return quickSelect(arr, low, pivotIndex-1, k);
        else
            return quickSelect(arr, pivotIndex+1, high, k);
    }
    return -1;
}
int main(){
    int arr[]={157,110,147,122,111,149,151,141,123,112,117,133};
    int n=sizeof(arr)/sizeof(arr[0]);
    int k;
    printf("CH.SC.U4CSE24142 - SANTHOSH A\n");
    printf("Enter k: ");
    scanf("%d", &k);
    int result=quickSelect(arr, 0, n-1, k-1);
    printf("k-th smallest element: %d\n", result);
    return 0;
}
```

OUTPUT:

```
D:\AVV CHENNAI\Semester 4\Design and Analysis of Algorithms\Lab Activities\Week 6>gcc Quickselect.c
D:\AVV CHENNAI\Semester 4\Design and Analysis of Algorithms\Lab Activities\Week 6>a
CH.SC.U4CSE24142 - SANTHOSH A
Enter k: 3
k-th smallest element: 112
```

WORKING:

LAB WEEK - 6 CH-SC-VAC-24/12
QUICK SELECT

Given: [157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133]
K = 3

STEP-1: Choose Pivot Element:
Pivot = 133 [Last Element]
We should rearrange the array based on the logic that
→ Elements < 133 to the left of 133 and → Elements > 133 to the right of 133.

STEP-2: Check the index
After partitioning:
[110, 122, 111, 123, 112, 117, 133, 141, 157, 151, 149]
The index of the pivot = 6
Pivot Index > K [6 > 2]
Since the pivot index is greater than k, left subarray is considered.
[110, 122, 111, 123, 112, 117]
New Pivot = 117
After partitioning:
[110, 111, 112, 117, 122, 123]
Pivot Index = 3 ; Pivot Index > K Index [3 > 2]
Consider left subarray.
[110, 111, 112]
New Pivot = 112
After partitioning:
[110, 111, 112]
Pivot Index = 2 ; Pivot Index = K Index [2 = 2]
3rd Smallest Number = 112.

Time Complexity: $O(N^2)$: Average case $O(n)$, but worst case $O(n^2)$ — because each call partitions the array like QuickSort, and in the worst case the pivot always splits poorly

Space Complexity: $O(N)$: $O(1)$ auxiliary space (in-place), but $O(N)$ worst-case recursion stack and $O(\log n)$ average recursion depth.